ОТЧЁТ

По лабораторной работе №2
«Tracerout»

Выполнил:
Студент гр. 410901
Зданович П.А.

Проверила:
Болтак С.В.

Цель работы: Изучить структуру UDP- и ICMP-пакетов, принцип действия поля TTL в IP-пакете, а также получить практические навыки сетевого программирования с использованием интерфейса сокетов при разработке аналога утилиты traceroute / tracert.

Условие задания: Необходимо разработать простейший аналог утилиты traceroute / tracert. В качестве транспортного протокола в данной работе был выбран UDP. Программа должна принимать в качестве параметра IP-адрес или имя целевого узла, выполнять трассировку маршрута и выводить промежуточные узлы аналогично системной утилите: номер хопа, время ожидания ответа для каждого отправленного пакета и адрес узла. Также требуется отправлять несколько пакетов на каждый шаг маршрута, сравнить результаты работы программы с системной утилитой tracert и проанализировать генерируемый трафик с помощью Wireshark. Дополнительно необходимо реализовать режим разрешения IP-адресов в имена узлов через обратные DNS-запросы.

Ход выполнения работы

В ходе выполнения лабораторной работы была разработана программа traceroute.py на языке Python. Программа использует UDP-сокет для отправки тестовых пакетов на целевой узел и raw ICMP-сокет для приема служебных ICMP-ответов от маршрутизаторов и конечного узла. Такой подход соответствует классическому принципу работы traceroute: для каждого значения TTL отправляется несколько UDP-пакетов на увеличивающиеся номера портов, а промежуточные маршрутизаторы возвращают ICMP-сообщения типа Time Exceeded при исчерпании TTL.

Алгоритм работы программы следующий. Сначала пользователь задает IP-адрес либо доменное имя узла назначения. Если в качестве аргумента указано доменное имя, программа получает соответствующий IP-адрес с помощью DNS-разрешения. Далее для каждого значения TTL, начиная с 1, создается UDP-сокет, у которого через setsockopt устанавливается текущее значение времени жизни пакета. Затем на адрес назначения отправляются три UDP-пакета. После каждой отправки программа ожидает ответ на raw ICMP-сокете и измеряет время между отправкой и получением ответа. Если ответ получен, определяется IP-адрес узла, который его отправил, и вычисляется

время отклика в миллисекундах. Если ответ не был получен за заданный интервал времени, в выводе отображается символ «*».

В реализованной программе предусмотрен также режим отображения имен узлов. Для этого используется обратный DNS-запрос по IP-адресу промежуточного маршрутизатора. Если имя для данного адреса найдено, в результатах выводится доменное имя и IP-адрес; если имя отсутствует, выводится только IP-адрес. Подобный режим работы соответствует поведению системных traceroute / tracert.

Рассмотрим работу программы для IP-адреса 8.8.8.8.

```
(.venv) bash-5.3$ sudo python main.py 8.8.8.8
Password:
Трассировка маршрута к 8.8.8.8 [8.8.8.8] с максимум 15 прыжков:
 1    4.28 ms 2.71 ms 6.22 ms    192.168.0.1
 2    4.34 ms 10.33 ms 7.17 ms    asr-dc.bn.by (212.98.160.250)
 3    4.34 ms 3.90 ms 19.52 ms    msq.asr-mck.bn.by (212.98.160.168)
 4    3.88 ms 10.09 ms 6.90 ms    185.11.76.41
 5    16.29 ms 25.41 ms 10.74 ms    185.11.76.26
 6    14.26 ms 15.92 ms *    74.125.49.252
 7    19.88 ms 100.49 ms 18.93 ms    108.170.234.167
 8    24.04 ms 20.29 ms 20.50 ms    108.170.234.245
 9    15.80 ms 19.47 ms 13.16 ms    dns.google (8.8.8.8)
```

Рисунок 1 – Результат выполнения traceroute.py

На данном этапе программа успешно построила маршрут до узла назначения. Для каждого промежуточного хопа были выведены три измерения времени отклика, а также IP-адрес маршрутизатора, от которого был получен ICMP-ответ. Видно, что количество хопов и характер задержек соответствуют ожидаемому результату работы утилиты трассировки маршрута.

Далее был выполнен анализ сетевого трафика, генерируемого программой, с помощью Wireshark.

icmp

No.	Time	Source	Destination	Protocol	Length	Info
239	14.949206	212.98.160.250	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
241	14.956920	212.98.160.250	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
243	14.961759	212.98.160.168	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
245	14.969232	212.98.160.190	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
247	14.998098	212.98.160.69	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
251	15.024350	185.11.76.41	192.168.0.213	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
253	15.028957	185.11.76.45	192.168.0.213	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
255	15.036894	185.11.76.41	192.168.0.213	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
257	15.050749	185.11.76.26	192.168.0.213	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
259	15.057662	185.11.76.26	192.168.0.213	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
261	15.061834	185.11.76.26	192.168.0.213	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
953	45.089332	64.233.174.229	192.168.0.213	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
959	45.197855	192.178.96.241	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
967	45.220498	108.170.234.167	192.168.0.213	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
980	45.284496	209.85.252.109	192.168.0.213	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
985	45.298857	209.85.252.109	192.168.0.213	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
987	45.310925	209.85.253.225	192.168.0.213	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
1018	45.498677	8.8.8.8	192.168.0.213	ICMP	70	Destination unreachable (Port unreachable)
1020	45.513641	8.8.8.8	192.168.0.213	ICMP	70	Destination unreachable (Port unreachable)
1027	45.525674	8.8.8.8	192.168.0.213	ICMP	70	Destination unreachable (Port unreachable)

Рисунок 2 – Вид трафика в wireshark при работе утилиты

В ходе анализа трафика было установлено, что программа действительно отправляет UDP-пакеты на целевой узел. Для каждого значения TTL передаются несколько UDP-запросов на порты, начиная с 33434 и далее увеличиваясь. В ответ от промежуточных маршрутизаторов поступают ICMP-пакеты типа Time Exceeded, что позволяет определить очередной узел маршрута. Когда пакет достигает конечного хоста, вместо Time Exceeded может быть получено ICMP-сообщение типа Destination Unreachable, свидетельствующее о достижении узла назначения и отсутствии приложения на соответствующем UDP-порту.

230	14.892814	192.168.0.213	8.8.8.8	UDP	46	64988 → 33434 Len=4
231	14.900276	192.168.0.1	192.168.0.213	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
232	14.900471	192.168.0.213	8.8.8.8	UDP	46	64988 → 33435 Len=4
233	14.907755	192.168.0.1	192.168.0.213	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
234	14.907939	192.168.0.213	8.8.8.8	UDP	46	64988 → 33436 Len=4
235	14.929884	192.168.0.1	192.168.0.213	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
236	14.932290	192.168.0.213	8.8.8.8	UDP	46	56992 → 33437 Len=4
237	14.940769	212.98.160.250	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
238	14.941078	192.168.0.213	8.8.8.8	UDP	46	56992 → 33438 Len=4
239	14.949206	212.98.160.250	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
240	14.949448	192.168.0.213	8.8.8.8	UDP	46	56992 → 33439 Len=4
241	14.956920	212.98.160.250	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
242	14.958308	192.168.0.213	8.8.8.8	UDP	46	49770 → 33440 Len=4
243	14.961759	212.98.160.168	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
244	14.961978	192.168.0.213	8.8.8.8	UDP	46	49770 → 33441 Len=4
245	14.969232	212.98.160.190	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
246	14.969480	192.168.0.213	8.8.8.8	UDP	46	49770 → 33442 Len=4
247	14.998098	212.98.160.69	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)

Рисунок 3 – Отправленные udp-пакеты в wireshark

На данном фрагменте можно увидеть исходящие UDP-пакеты, формируемые программой. Для каждого следующего шага маршрута значение TTL увеличивается на единицу. Также видно, что на один и тот же TTL отправляется несколько пакетов, что позволяет получить несколько измерений времени отклика и сделать результат более наглядным.

231	14.900276	192.168.0.1	192.168.0.213	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
233	14.907755	192.168.0.1	192.168.0.213	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
235	14.929884	192.168.0.1	192.168.0.213	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
237	14.940769	212.98.160.250	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
239	14.949206	212.98.160.250	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
241	14.956920	212.98.160.250	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
243	14.961759	212.98.160.168	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
245	14.969232	212.98.160.190	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
247	14.998098	212.98.160.69	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
251	15.024350	185.11.76.41	192.168.0.213	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
253	15.028957	185.11.76.45	192.168.0.213	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
255	15.036894	185.11.76.41	192.168.0.213	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
257	15.050749	185.11.76.26	192.168.0.213	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
259	15.057662	185.11.76.26	192.168.0.213	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
261	15.061834	185.11.76.26	192.168.0.213	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
953	45.089332	64.233.174.229	192.168.0.213	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
959	45.197855	192.178.96.241	192.168.0.213	ICMP	110	Time-to-live exceeded (Time to live exceeded in transit)
967	45.220498	108.170.234.167	192.168.0.213	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
980	45.284496	209.85.252.109	192.168.0.213	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)
985	45.298857	209.85.252.109	192.168.0.213	ICMP	74	Time-to-live exceeded (Time to live exceeded in transit)

Рисунок 4 – ICMP-пакеты типа Time Exceeded

ICMP-сообщения типа Time Exceeded подтверждают, что маршрутизатор уменьшил TTL до нуля и отбросил пакет, после чего отправил уведомление

источнику. Именно по IP-адресу источника такого ICMP-пакета программа определяет очередной промежуточный узел маршрута.

1018	45.498677	8.8.8.8	192.168.0.213	ICMP	70	Destination unreachable (Port unreachable)
1020	45.513641	8.8.8.8	192.168.0.213	ICMP	70	Destination unreachable (Port unreachable)
1027	45.525674	8.8.8.8	192.168.0.213	ICMP	70	Destination unreachable (Port unreachable)

Рисунок 5 – ICMP-пакет типа Destination Unreachable от узла назначения

После достижения пакетом узла назначения появляется ICMP-ответ типа Destination Unreachable. Это означает, что сам узел уже достигнут, однако указанный UDP-порт не используется приложением. Для traceroute на UDP это является штатным признаком завершения трассировки.

Отдельно была выполнена проверка корректности результатов путем сравнения с системной утилитой traceroute.

```
traceroute to 8.8.8.8 (8.8.8.8), 64 hops max, 40 byte packets
 1  192.168.0.1 (192.168.0.1)  3.923 ms  3.170 ms  3.781 ms
 2  asr-dc.bn.by (212.98.160.250)  4.039 ms  5.612 ms  4.476 ms
 3  msq.asr-km.bn.by (212.98.160.190)  4.018 ms
    msq.asr-mck.bn.by (212.98.160.69)  5.622 ms
    msq.asr-km.bn.by (212.98.160.190)  4.666 ms
 4  185.11.76.41 (185.11.76.41)  4.842 ms
    185.11.76.22 (185.11.76.22)  5.145 ms
    185.11.76.41 (185.11.76.41)  4.560 ms
 5  185.11.76.26 (185.11.76.26)  5.567 ms  4.592 ms  4.481 ms
 6  * 74.125.49.252 (74.125.49.252)  14.578 ms  11.578 ms
 7  192.178.97.125 (192.178.97.125)  15.537 ms
    216.239.58.135 (216.239.58.135)  12.784 ms
    192.178.97.13 (192.178.97.13)  12.981 ms
 8  108.170.234.103 (108.170.234.103)  12.628 ms
    142.250.238.1 (142.250.238.1)  12.971 ms
    209.85.253.225 (209.85.253.225)  11.380 ms
 9  dns.google (8.8.8.8)  11.339 ms  10.559 ms  11.339 ms
```

Рисунок 6 – Результат работы системной утилиты traceroute для 8.8.8.8

Сравнение показало, что разработанная программа и системная утилита выводят сходный маршрут до узла назначения. Количество промежуточных узлов и порядок их прохождения в целом совпадают. Незначительные отличия во времени отклика объясняются текущим состоянием сети, изменением загрузки каналов, а также тем, что сетевые задержки являются непостоянной величиной.

Следующим этапом была проверена работа программы при указании доменного имени вместо IP-адреса. В качестве примера был использован домен google.com.

```
(.venv) bash-5.3$ sudo python main.py google.com
Password:
Трассировка маршрута к google.com [142.250.181.238] с максимум 15 прыжков:
 1    7.85 ms 7.31 ms 7.04 ms    192.168.0.1
 2    12.05 ms 6.11 ms 7.17 ms    asr-dc.bn.by (212.98.160.250)
 3    3.97 ms 13.37 ms 3.12 ms    msq.asr-mck.bn.by (212.98.160.168)
 4    9.62 ms 4.37 ms 5.12 ms    185.11.76.41
 5    4.86 ms 4.19 ms 17.89 ms    185.11.76.26
 6    * * *
 7    17.15 ms 15.83 ms 12.33 ms    216.239.58.135
 8    21.56 ms 12.75 ms 16.28 ms    216.239.58.78
 9    27.42 ms 23.84 ms 22.49 ms    192.178.83.133
10    21.31 ms 24.39 ms 25.41 ms    216.239.43.94
11    21.64 ms 20.21 ms 21.64 ms    192.178.106.77
12    29.32 ms 21.37 ms 21.06 ms    142.251.241.83
13    26.91 ms 20.84 ms 26.97 ms    lcberc-ab-in-f14.1e100.net (142.250.181.238)
```

Рисунок 7 – Результат работы traceroute.py для доменного имени google.com

При запуске программы с доменным именем сначала выполняется DNS-разрешение имени узла в IP-адрес, после чего трассировка маршрута производится уже до полученного IP-адреса. Таким образом, программа поддерживает оба режима запуска: как по IP-адресу, так и по доменному имени, что соответствует условию задания.

141	6.866749	192.168.0.213	192.168.0.1	DNS	70	Standard query 0x11c6 A google.com
143	6.944138	192.168.0.1	192.168.0.213	DNS	86	Standard query response 0x11c6 A google.com A 142.250.181.238

Рисунок 8 – DNS-запрос в Wireshark при указании доменного имени

В Wireshark при этом можно наблюдать DNS-запрос, необходимый для определения IP-адреса узла назначения. После получения ответа дальнейшая работа программы протекает так же, как и в случае прямого указания IP-адреса.

Также была проверена работа дополнительного режима разрешения адресов промежуточных узлов в доменные имена.

```
5.99 ms 14.77 ms 8.94 ms    asr-dc.bn.by (212.98.160.250)
3.96 ms 4.29 ms 3.70 ms    msq.asr-km.bn.by (212.98.160.190)
```

Рисунок 9 – Работа программы в режиме разрешения имен узлов

В данном режиме программа для каждого IP-адреса промежуточного маршрутизатора пытается выполнить обратный DNS-запрос. Если PTR-запись существует, то в выводе отображается доменное имя и IP-адрес узла. Если запись отсутствует, программа продолжает корректно работать и выводит только IP-адрес. Это делает результат более информативным и приближает его к поведению стандартных сетевых утилит.

9113	357.937536	192.168.0.1	192.168.0.213	DNS	309	Standard query response 0xa8aa A main.vscode-cdn.net CNAME vscode-cdn.z01.azurefd.net CNA
9115	357.946611	192.168.0.213	192.168.0.1	DNS	94	Standard query 0xfc2b A p2p-fra2.discovery.steamserver.net
9116	357.951892	192.168.0.1	192.168.0.213	DNS	158	Standard query response 0xfc2b A p2p-fra2.discovery.steamserver.net A 155.133.226.75 A 15
9220	360.539773	192.168.0.213	192.168.0.1	DNS	86	Standard query 0x9793 PTR 252.49.125.74.in-addr.arpa
9221	360.635082	192.168.0.1	192.168.0.213	DNS	165	Standard query response 0x9793 No such name PTR 252.49.125.74.in-addr.arpa SOA ns1.google
9228	360.684691	192.168.0.213	192.168.0.1	DNS	87	Standard query 0xe5e8 PTR 241.96.178.192.in-addr.arpa
9229	360.840412	192.168.0.1	192.168.0.213	DNS	167	Standard query response 0xe5e8 No such name PTR 241.96.178.192.in-addr.arpa SOA ns1.googl
9236	360.903150	192.168.0.213	192.168.0.1	DNS	85	Standard query 0xa0af PTR 17.237.14.72.in-addr.arpa
9237	361.079044	192.168.0.1	192.168.0.213	DNS	145	Standard query response 0xa0af No such name PTR 17.237.14.72.in-addr.arpa SOA ns1.google.

Рисунок 10 – Обратные DNS-запросы (PTR) в Wireshark

В анализе трафика Wireshark можно заметить появление дополнительных DNS-запросов обратного разрешения имен. Это подтверждает корректную реализацию дополнительного задания, связанного с определением доменных имен промежуточных узлов.

В ходе тестирования было установлено, что корректность работы программы зависит от настроек сетевого экрана. Если фаервол блокирует ICMP-пакеты типа Time Exceeded или Destination Unreachable, определение промежуточных узлов становится невозможным. Поэтому во время тестирования требуется либо временно отключить сетевой экран, либо разрешить прохождение соответствующих ICMP-пакетов.

Вывод

В ходе выполнения лабораторной работы был разработан аналог утилиты traceroute / tracert на языке Python с использованием интерфейса сокетов. В качестве способа передачи данных использовались UDP-пакеты, а определение промежуточных узлов маршрута осуществлялось по входящим ICMP-сообщениям. Программа поддерживает трассировку как по IP-адресу, так и по доменному имени, отправляет несколько пакетов для каждого значения TTL, измеряет время отклика и выводит узлы маршрута в удобной форме.

С помощью Wireshark было подтверждено, что программа формирует UDP-пакеты с возрастающим значением TTL, получает ICMP Time Exceeded от промежуточных маршрутизаторов и ICMP Destination Unreachable от узла назначения. Также была проверена работа режима разрешения имен узлов через обратные DNS-запросы. Сравнение с системной утилитой tracert показало, что разработанная программа корректно выполняет поставленную задачу и может рассматриваться как